QUERIOUS AI

A Transparent, Self-Correcting Natural-Language Interface for Data Analysis

Software Design Document (SDD)

CSCI 5520 - AI-Driven Software Development

Student Name	Edel Kiprop
Date	October 6, 2026
Document Version	v1.0 (Final)
Project Repository / URL (if available)	N/A

1 Problem & Users

1.1 Problem Description

Many people who need answers from data cannot get those answers themselves. Managers, analysts, students, and small-business owners regularly have simple questions such as "Which products sold best last quarter?" or "How many customers did we lose in each region?" Answering these questions usually requires writing SQL or Python, so non-technical users either wait for a technical colleague, work through spreadsheets by hand, or give up on the question. This creates delays, bottlenecks around a few technical people, and decisions made without evidence.

Large language models (LLMs) can now translate plain-English questions into database queries, and general-purpose AI agents can be given tools to run code and query data on their own. However, these approaches introduce a new problem: **the user usually cannot tell whether the answer is correct.** An AI may misread an ambiguous term, choose the wrong column, join tables incorrectly, or produce a confident number that is simply wrong. Autonomous agents make this worse because the model decides what to do, how to do it, and when it is finished, and the same question can follow a different path and produce a different result each time. Their reasoning is often hidden or presented as a long log that a non-technical user cannot interpret.

In data analysis, a wrong answer that looks right is more harmful than no answer, because people act on the numbers they see. The core problem is therefore the lack of a way for non-technical people to ask questions about their data in plain English and receive answers that are **correct, consistent, and verifiable**, without needing the technical skills to check the work themselves.

1.2 Target Users & Stakeholders

User / Stakeholder	Relationship to the Problem
Primary User - Business User / Non-Technical Analyst	Has questions about a dataset but cannot write SQL or code. Needs answers quickly, needs to understand how each answer was found, and needs to know when the system is unsure or cannot answer.
Data Steward	Understands the dataset and what business terms mean (for example, what counts as an "active customer"). Defines terms and table relationships in the semantic layer so that the system interprets questions consistently.
Decision-Makers / Report Consumers	Do not use the system directly, but act on its answers. Care that numbers are accurate, reproducible, and explainable if questioned.
Developer / Course Evaluator	Builds, tests, and evaluates the system. Needs measurable evidence (accuracy, correction rate, consistency, latency) that the design works and that it differs meaningfully from a general AI agent.
IT / Data Governance	Stakeholders are concerned with data privacy, what data is sent to third-party AI services, credential handling, and preventing any modification of source data.

2 Proposed Solution & Scope

2.1 Proposed Solution

Querious AI is a web application that lets users ask questions about a dataset in plain English and receive an answer together with a clear, plain-language explanation of exactly how that answer was

found. The user can see how their question was interpreted, which business definitions were applied, the query that was run, which checks the result passed, and any corrections the system made along the way.

The key design idea is **"the AI suggests, the system checks."** Unlike a general-purpose AI agent, where the language model makes every decision, Querious limits the AI to two narrow jobs: deciding whether a question is clear enough to answer, and turning the question into a structured query plan. Everything after that is handled by ordinary, testable software: the plan is checked against the real data structure and the agreed business definitions, converted into a database query in a predictable way, run with read-only access, and checked for results that do not make sense. When a step fails, the system sends the exact problem back to the AI to fix, with a strict limit on retries. If the question is unclear, Querious asks the user instead of guessing; if it cannot produce a trustworthy answer, it says so instead of inventing one.

The expected value is that non-technical users can get answers from data on their own while being able to trust, check, and explain those answers. Compared with an autonomous AI agent, Querious trades open-ended flexibility for correctness, consistency, safety, and transparency, which are the qualities that matter most when people make decisions from numbers.

The trust layer reports observable evidence rather than a probability that an answer is correct. A deterministic Confidence/Trust Score, query risk classification, answer provenance, and correction replay make verification visible. Selectable interpretations, capability previews, and schema-aware follow-up questions help users stay within supported analysis. Passing checks does not establish that the data are complete or that the user selected the intended meaning.

2.2 Minimum Viable Product (MVP) Scope

The MVP is a single-user web application running on a local machine that demonstrates the full question-to-verified-answer pipeline on tabular data.

- Load a dataset by uploading a CSV file or selecting a provided sample dataset, and view its tables, columns, and defined business terms.
- Ask a question in plain English and receive an answer as a results table, a short plain-English summary, and a simple chart when the result shape supports one.
- Ambiguity detection with selectable alternative interpretations and definitions, a Continue action, and a “None of these, rephrase” exit before execution.
- A structured query plan (intermediate representation, IR) produced by the LLM and validated against the dataset schema and semantic layer before execution.
- Deterministic compilation of the validated plan into a single read-only SQL query.
- Safe execution against an embedded, read-only database with a query timeout and result-size limit.
- Result sanity checks (for example: empty results, impossible percentages, unexpected row multiplication after joins).
- Bounded self-correction: failed validation or failed checks are sent back to the planner with the exact error, up to three attempts.
- A “How I got this” panel showing the interpretation, terms, plan, SQL, risk class, verification evidence, answer provenance, and a replay of correction attempts.
- An honest failure state that explains why a question could not be answered and suggests how to rephrase it.
- A semantic layer configuration file in which a data steward defines business terms and allowed table relationships.
- A persistent audit log and question history that allows past answers and their audit trails to be reopened.
- Simple answer feedback (mark as correct or incorrect) stored with the audit record.
- Deterministic Confidence/Trust Score expressed as High, Medium, or Low verification support with the exact reasons; no LLM-generated confidence percentages.

- A capability/difficulty preview listing supported operations and deterministic risk class before execution, with stronger checks for higher-risk plans and early rejection of unsupported requests.
- Answer provenance identifying the source dataset(s), referenced columns, rows considered, and applied filters, plus up to three schema-validated suggested follow-up questions.
- A core evaluation harness that runs a gold set of questions and reports accuracy, clarification quality, correction rate, consistency, latency, and trust-layer acceptance checks (US-13).

Supporting capabilities required: access to a third-party LLM API with the key held only on the server, an embedded analytical database, and local storage for the audit log.

Evaluation/research enhancement: US-14 adds an optional developer-only baseline comparison mode after the core MVP is complete. It compares direct LLM-to-SQL with the constrained Querious pipeline using the same gold set. It is not a normal user feature or a prerequisite for the core demonstration. Public benchmark expansion and ablation studies remain future work.

2.3 Out of Scope / Non-Goals

- **Writing or changing data.** Querious never inserts, updates, deletes, or alters data or schemas. It is strictly a read-only question-answering tool.
- **Autonomous multi-step investigation.** Querious answers one question at a time with a fixed pipeline. It does not plan open-ended research tasks, browse the web, or combine data with outside sources.
- **Arbitrary SQL features.** The MVP query plan supports filters, aggregations, grouping, sorting, top-N, time bucketing, and joins declared in the semantic layer. Window functions, nested subqueries, and user-written SQL are not supported.
- **Direct connections to production databases or cloud warehouses** (PostgreSQL servers, Snowflake, BigQuery, etc.). The MVP uses uploaded files loaded into an embedded database.
- **Multi-user accounts, authentication, and sharing.** The MVP is a single-user local application.
- **A graphical editor for the semantic layer.** Terms are defined in a configuration file; the app shows them read-only.
- **Predictive analytics, forecasting, or machine-learning models** built from the user's data.
- **Dashboards, scheduled reports, or export to BI tools.** A read-only board of pinned, verified answers is noted as future work (FUT-01, Section 7.2).
- **Fine-tuning or hosting a custom language model.**

3 User Stories & Acceptance Criteria

The following stories define the major behavior required for the MVP, except US-14, which is an evaluation/research enhancement. Stable identifiers trace into the UI/UX design, implementation plan, GitHub Issues, tests, and agentic development work. The gold question set is the fixed set of questions with known answers used by US-13; the optional paired comparison is defined in US-14.

3.1 Business User / Analyst Stories

US-01 - Load a Dataset

User Story: As a business user, I can upload a CSV file or select a provided sample dataset so that I can ask questions about data that matters to me.

Pass/Fail Check: Pass if a CSV file with a header row and a size up to 50 MB loads and its table and columns are displayed within 10 seconds, and a file that is not a valid CSV is rejected with a clear message; fail if an invalid file is accepted or a valid file fails silently.

US-02 - View Available Data and Terms

User Story: As a business user, I can see the tables, columns, example values, and defined business terms in my dataset so that I know what kinds of questions I can ask.

Pass/Fail Check: Pass if the data overview lists every loaded table and column with its data type and lists every term defined in the semantic layer with its plain-English description.

US-03 - Ask a Question in Plain English

User Story: As a business user, I can type a question in plain English and receive an answer as a table and short summary so that I can get information without writing code.

Pass/Fail Check: Pass if, for questions in the gold question set marked as answerable, the system returns a results table and summary, and the result matches the known correct answer for at least the target accuracy agreed in the evaluation plan (initial target: 80%); fail if answers are returned without an executed query behind them.

US-04 - Clarify Ambiguous Questions

User Story: As a business user, I can compare and select alternative interpretations so that the system does not silently guess my meaning.

Pass/Fail Check: Pass if each ambiguous gold-set case presents two or three schema-valid alternatives with their measure, grouping, or time-period definition before any query runs; Continue is disabled until a choice is selected. “None of these, rephrase” executes nothing. The selected interpretation is used in the plan, displayed with the answer, and retained in history.

US-05 - See How the Answer Was Found

User Story: As a business user, I can inspect verification support and provenance in “How I got this” so that I can check and explain the result.

Pass/Fail Check: Pass if every answer displays its deterministic trust level and reasons, interpretation, terms, plan, risk class, executed SQL, checks, and source dataset(s), columns, rows considered, and filters. SQL and provenance must match the execution/audit record. Fixed signal fixtures must produce exactly the RC-17 trust level; no confidence percentage may appear.

US-06 - Automatic, Visible Correction

User Story: As a business user, I can replay failed and corrected attempts so that I understand what happened.

Pass/Fail Check: Pass if an injected first-attempt schema failure is recovered within three total planning attempts and replay shows each attempt number, failed stage, error, revised plan change, and final executed SQL/check outcome. Reopening history reproduces the same sequence without rerunning queries. Failed attempts must never display a result as an answer.

US-07 - Honest Failure

User Story: As a business user, I can see the supported capability and difficulty before execution and receive an honest failure when needed.

Pass/Fail Check: Pass if supported plans show operations and RC-18 risk before execution. Known unsafe, missing-data, or unsupported requests stop before compilation/execution with a reason and a schema-valid rephrasing suggestion; requests discovered unsupported during planning stop at validation. After three failed attempts, no answer or confidence score is displayed. An execution spy confirms zero executions for early-rejected cases.

US-08 - Review Question History

User Story: As a business user, I can reopen my previous questions and their answers so that I can revisit or share how a result was obtained.

Pass/Fail Check: Pass if previously asked questions appear in a history list after restarting the application and selecting one shows the original answer and its full audit trail.

US-09 - Give Feedback on an Answer

User Story: As a business user, I can mark an answer as correct or incorrect so that answer quality can be tracked and improved.

Pass/Fail Check: Pass if selecting correct or incorrect stores the rating with the matching audit record and the rating is visible when the answer is reopened from history.

US-10 - Schema-Aware Follow-Up Questions (Core MVP)

User Story: As a business user, I can choose a supported follow-up after an answer so that I can continue exploring the available data.

Pass/Fail Check: Pass if up to three suggestions are produced from schema/semantic-layer templates and the final plan, every referenced column/term and operation validates, and selecting one starts a new question through the full pipeline. If no valid suggestion exists, none is shown. Missing-column and forecasting fixtures must produce no unsupported suggestions.

US-11 - Risk-Based Verification (Core MVP)

User Story: As a business user, I can see the query risk and additional checks so that the system gives complex supported questions stronger verification.

Pass/Fail Check: Pass if fixed low/medium/high plan fixtures receive RC-18 classifications and all mandatory checks for their class run before an answer is released. A failed join-cardinality check blocks the answer and triggers bounded correction; an unavailable mandatory check yields honest failure, never a verified answer.

3.2 Data Steward Stories

US-12 - Define Business Terms and Relationships

User Story: As a data steward, I can define business terms and allowed table relationships in a configuration file so that questions are interpreted consistently with how my organization defines them.

Pass/Fail Check: Pass if adding a term (for example, "active customer = customer with an order in the last 90 days") causes questions using that phrase to apply the definition and show it in the audit trail, and if a definition that refers to a column that does not exist is rejected at load time with an error naming the problem.

3.3 Developer / Evaluator Stories

US-13 - Run the Evaluation Suite (Core MVP)

User Story: As the developer, I can run the gold question set with one command to measure whether the design works.

Pass/Fail Check: Pass if the report gives execution accuracy against known results, clarification precision/recall, correction success among initially failed recoverable cases, repeated-run consistency, median/p90 end-to-end latency, and calls per question. It must also report results of RC-17–RC-22 acceptance fixtures and checks by risk class. The initial answerable-question accuracy target is 80%; it is a target, not an observed result.

US-14 - Compare Against Direct LLM-to-SQL (Evaluation/Research Enhancement)

User Story: As the evaluator, I can run a paired comparison between direct LLM-to-SQL and Querious so that I can measure the effects of the constrained pipeline.

Pass/Fail Check: Pass if a developer command runs both conditions on identical frozen inputs and exports per-question outcomes and aggregate accuracy, invalid-query rate, consistency, latency, model calls, and cost when available. The report identifies model/configuration, prompts, versions, budgets, seeds where supported, and denominators. Neither unsafe SQL nor baseline outputs appear as trusted user answers.

4 Requirements, Constraints & Design Rules

The following rules apply across individual user stories and must be respected consistently during implementation, including by AI development agents.

Safety & Correctness

- **RC-01 - Read-Only Data Access:** The database connection used for answering questions must be opened in read-only mode. Only a single SELECT statement may be executed per attempt; any statement containing data-changing or schema-changing operations must be rejected before execution. Deterministic provenance/reconciliation helper SELECTs are the limited instrumentation exception described in RC-23; each is separately checked and is not LLM-generated SQL.
- **RC-02 - LLM Output Is Never Executed Directly:** Text produced by the LLM must never be run as SQL or code. LLM output must first parse into the defined query-plan schema, pass validation, and be compiled to SQL by the deterministic compiler.

- **RC-03 - No Fabricated Answers:** Every number or value shown to the user must come from an executed query result. Summaries must be generated from the validated plan and actual results, not from free-form LLM statements about the data.
- **RC-04 - Bounded Self-Correction:** Each question may make at most three planning attempts. Each retry must include the specific validation error, database error, or failed check. After the limit, the system must show the honest failure state (US-07).
- **RC-05 - Declared Joins Only:** The planner may only join tables using relationships declared in the semantic layer. Undeclared joins must fail validation.
- **RC-06 - Determinism:** The same validated plan must always compile to identical SQL. LLM calls should use the lowest-randomness settings available, and every plan must be stored so results can be reproduced.

Security & Privacy

- **RC-07 - Data Minimization to the LLM:** Only schema metadata (table and column names and types), semantic-layer definitions, the user's question, and error messages may be sent to the LLM. Full data rows must never be sent. A small number of example values per text column (maximum five) may be sent only when enabled in configuration.
- **RC-08 - Secrets Handling:** The LLM API key must be stored as a server-side environment variable. It must never appear in front-end code, logs, or the repository.
- **RC-09 - Auditability:** Every question must produce an audit record containing timestamps, the question, clarification choices, each plan attempt, the SQL executed, errors, checks performed and their results, a result summary, user feedback, and the LLM model identifier used. Also retain dataset fingerprint/version, schema and semantic-layer versions, capability result, risk-policy version, trust-policy version and signals, provenance, replay events, and required checks that passed, failed, or were unavailable.

Performance & Reliability

- **RC-10 - Execution Limits:** Each query must time out after 15 seconds, and at most 5,000 result rows are returned to the interface.
- **RC-11 - Responsiveness:** Under normal network conditions, a typical question should complete within 20 seconds end-to-end. The interface must show the current stage (understanding, planning, checking, running) while work is in progress.
- **RC-12 - LLM Failure Handling:** If the LLM service is unavailable or rate-limited, the system must show a clear service error rather than hang or show a partial answer.

Accessibility & User Experience

- **RC-13 - Plain Language First:** User-facing text must use plain business language by default. Technical details such as SQL and the raw plan appear only inside the expandable "How I got this" panel.
- **RC-14 - Accessibility:** The interface should meet WCAG 2.1 AA basics: full keyboard navigation, sufficient color contrast, labels for screen readers, and status not conveyed by color alone. Every chart must be accompanied by the underlying table.

Platform & Compatibility

- **RC-15 - Supported Clients:** The MVP targets current desktop versions of Chrome, Firefox, and Edge. Mobile layouts are not required.
- **RC-16 - Swappable LLM Provider:** All LLM calls must go through a single provider-interface module so that the model or provider can be changed without modifying pipeline logic.
- **RC-17 - Deterministic Confidence/Trust Score:** The backend computes an ordinal verification-support level from recorded signals, never from an LLM assertion or percentage. For released answers, High requires an unambiguous or explicitly selected interpretation, successful schema/semantic validation, compilation and execution, all risk-required checks passed, complete

provenance, no correction, and no non-blocking warning. Medium requires the same mandatory passes but at least one corrected attempt. Low applies when mandatory checks passed but a documented non-blocking warning remains, such as empty results or truncated display; it overrides Medium. Failed or unavailable mandatory checks, unresolved ambiguity, or incomplete provenance block the answer and show “Not available” rather than a score. The level is not a calibrated probability of correctness; policy and signals are stored.

- **RC-18 - Query Risk and Stronger Verification:** Classify the validated IR by deterministic rules, choosing the highest applicable class: Low = single-table projection, filter, or count; Medium = other aggregation, grouping, sorting/top-N, or time bucketing; High = any declared join, multiple derived transformations, or a question that required semantic clarification. All classes require schema/term validation, read-only safety, execution limits, and applicable result checks. Medium adds aggregate-type, null-handling, grouping-grain, and date-boundary checks as applicable. High adds declared-key/cardinality and row-multiplication checks for joins, plus independent aggregate reconciliation or filter/definition checks for the triggering operation. Mandatory checks and applicability are logged; unsupported operations are rejected regardless of risk.
- **RC-19 - Provenance:** Build provenance from the validated plan and execution metadata, not LLM prose. Record source file/table identifiers and dataset fingerprint, columns used (including filter/join columns), applied predicates and date boundaries, and the count of input records after joins/filters but before grouping or top-N. Label that count as “rows considered”; for joins also show per-source contributing-row counts so it is not mistaken for distinct source records. Record returned rows and display truncation separately. Never infer input count from result-table length.
- **RC-20 - Capability Preview and Alternatives:** Before the answer query executes, show supported operations, risk class, and a short reason for difficulty; do not promise accuracy or latency. Reject known unsupported requests at understanding and any newly discovered unsupported plan at validation. Clarification alternatives must use available columns and steward definitions; user selection is recorded and revalidated.
- **RC-21 - Suggested Follow-Ups:** Generate at most three suggestions locally from schema-aware templates, semantic definitions, and the final validated plan. Validate every referenced column, term, relationship, and operation before display. A suggestion is a new question and must pass clarification, validation, compilation, execution, and checks; suggestions must not claim causal explanations, forecasts, or unavailable data.
- **RC-22 - Correction Replay:** Derive the attempt timeline from immutable audit events: attempt number, stage, failure reason, revised plan change, and final outcome. Display SQL only when it was actually executed. Distinguish “not executed”, “executed but failed checks”, and “answer released”. Replay is a view of recorded evidence, not another model call or execution.
- **RC-23 - Isolated Baseline Evaluation (Research Enhancement):** Direct LLM-generated SQL may be parsed and executed only by a separate developer-only evaluation runner against a disposable copy of the evaluation dataset, with single-SELECT AST allowlisting, read-only access, timeouts, and output limits. This explicit experimental exception to RC-02 never enables raw SQL in the production question endpoint. Provenance/check helper SELECTs are deterministic internal instrumentation, independently safety-checked and subject to the same limits; they do not expand the IR or the three-planning-attempt budget.

5 Platform & Architecture

5.1 System Components & Technology Choices

Querious consists of four groups of components. A **web interface** in the user's browser handles asking questions, clarifications, answers, and audit trails. A **Python backend** runs the question pipeline: two LLM-assisted steps (ambiguity check and query planning) followed by deterministic steps (plan validation, SQL compilation, safe execution, result checks, explanation building, and audit logging). A **local data layer** holds the user's dataset in an embedded analytical database, the semantic-layer

configuration, and the audit store. Finally, a **third-party LLM API** is called only by the backend and only with schema-level information.

The deterministic backend adds a capability gate, risk-policy module, trust-policy module, provenance builder, and schema-aware suggestion templates. These are small functions over the existing IR, execution metadata, and audit events rather than additional agents or services. SQLite retains the evidence required for replay. The optional baseline runner is a separate developer command and is not reachable through the business-user API.

Technology	Rationale	Usage	Residency
React + TypeScript (Vite)	Widely used, well documented, and well supported by AI coding tools; TypeScript types reduce interface errors.	Implements the upload, data overview, question, clarification, answer, audit-trail, history, and feedback screens.	User's browser (served locally).
Python + FastAPI	Python has the strongest data and LLM ecosystem; FastAPI provides typed JSON APIs with little boilerplate.	Hosts the question pipeline and exposes API endpoints to the web interface.	Local machine.
Anthropic Claude API (leading option)	Strong at structured output and instruction following; supports schema-constrained responses.	Ambiguity check and translation of the question into the query-plan schema. Accessed only through the provider interface (RC-16).	Third-party cloud service.
Pydantic	Defines data schemas in Python and validates data against them with clear error messages.	Defines the query-plan (IR) schema; parses and validates LLM output; produces precise errors for retries.	Local machine (backend library).
SQLGlot	Python library for building, parsing, and analyzing SQL programmatically.	Compiles validated plans into SQL deterministically; parses SQL to confirm a single read-only SELECT.	Local machine (backend library).
DuckDB	Embedded analytical database that loads CSV files directly, needs no server, and supports read-only connections.	Stores and queries the user's dataset under read-only access with timeouts.	Local machine (in-process).
YAML semantic layer	Human-readable, version-controllable configuration.	Defines business terms, metrics, and allowed joins; validated against the schema at load time.	Local file in the project.
SQLite	Zero-configuration file database included with Python.	Stores audit records, question history, and feedback.	Local machine (file).
Recharts	Simple React charting library.	Renders bar and line charts when the result shape supports them.	User's browser.
pytest + evaluation harness	Standard Python testing framework.	Unit tests for validator, compiler, and checks; runs the gold question set and produces the evaluation report.	Developer machine / CI.

5.2 Architecture Diagram & Explanation

Figure 1 shows the Querious architecture. The browser communicates with the FastAPI backend over HTTPS/JSON. Inside the backend, a question flows down the left column and then down the right column. Only the two purple components call the third-party LLM API, and they send only schema-level information and business-term definitions, never full data rows (RC-07). The validated plan is compiled to SQL deterministically and run by the safe executor against the read-only DuckDB dataset. The results then pass sanity checks before the explanation builder produces the plain-English summary and audit view from the plan itself.

The dashed correction loop is the central design decision. When plan validation or result checks fail, the specific error is returned to the query planner for another attempt, up to three times (RC-04). Every step, including failures and corrections, is written to the SQLite audit store. The semantic layer is read by the planner, validator, and result checks so that business definitions and allowed joins are applied consistently. This structure is what distinguishes Querious from an autonomous AI agent: the model proposes, and deterministic, testable code decides what is executed and what is shown.

Figure 1 retains the core topology; its validator includes the capability gate and risk classifier, result checks apply the risk-specific policy, and the explanation/audit builder attaches trust reasons, provenance, replay events, and follow-up templates. A preview is sent to the browser after a supported plan validates and before the answer query executes. Each corrected plan is revalidated and reclassified. Deterministic count/reconciliation helper queries may be required for provenance and higher-risk verification; they share the read-only safety gate and limits. The normal pipeline is plan → validate → compile → execute → check → bounded correction when needed → answer. The separate research runner is not part of Figure 1's user path.

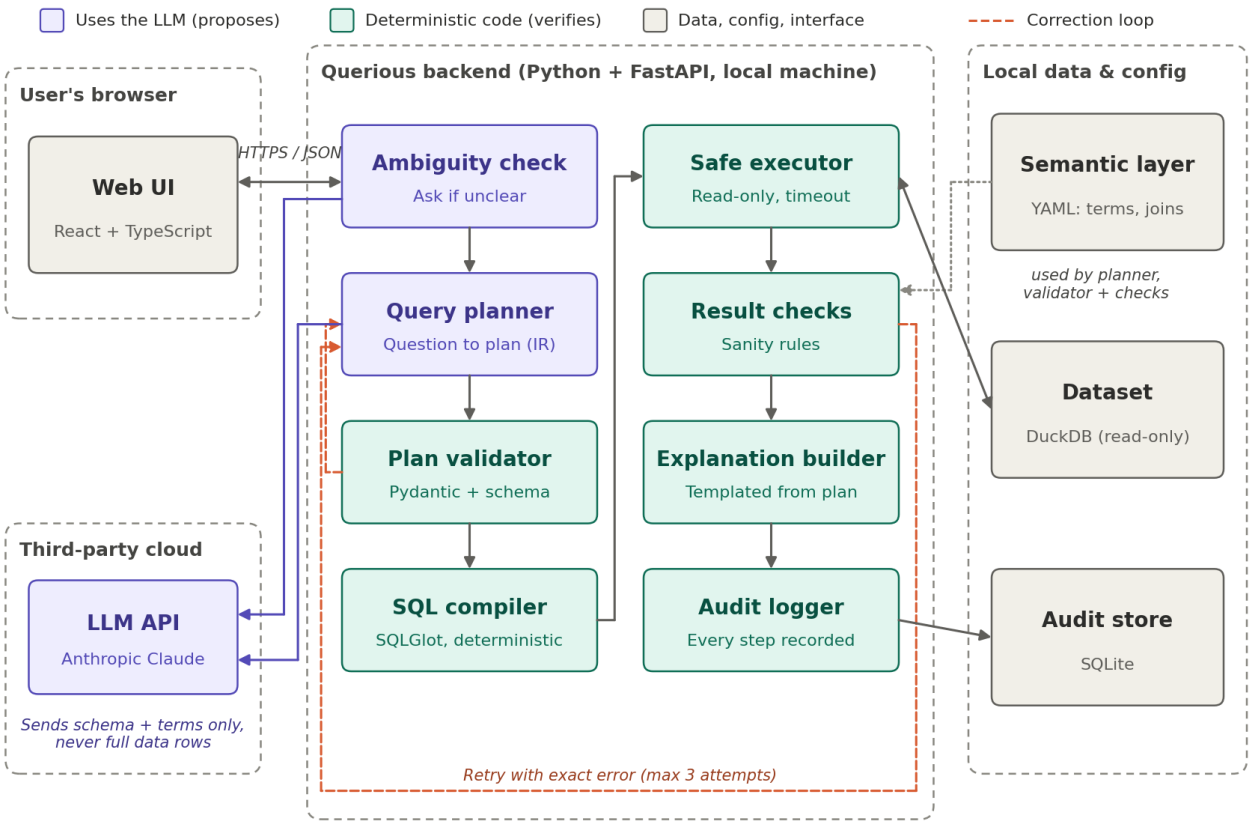


Figure 1: Architecture of Querious AI, showing LLM-assisted components (purple), deterministic verification components (teal), and the bounded correction loop.

6 UI/UX Design

Section 6 was produced with **Claude Design** (the Claude Design canvas) as the AI-assisted UI/UX design tool, used in place of Figma Make; the template permits another instructor-approved AI-assisted design workflow. The substantially completed SDD from Challenge #1 was given to Claude as project context. The first AI-generated prototype was treated as a draft: it was reviewed screen by screen against the user stories (US-##) and requirements (RC-##) and then refined. When this SDD was revised to v0.2, the same prototype was extended with the trust-layer requirements (US-10, US-11, RC-17 to RC-22) so that the prototype and the written requirements stay consistent. The design canvas has four pages: “6.1 Experience flow”, “6.2 Prototype (refined)”, “6.3 AI draft v1”, which keeps the unrevised first draft for comparison, and “Layout options”, which keeps the three layout directions compared before the final layout was chosen.

6.1 Application / Experience Flow

Querious has one primary user experience for the business user. After loading a dataset (1) and reviewing the available data and business terms (2), the user asks a question (3), either by typing it or by picking one of the suggested questions built from the loaded dataset’s columns and business terms. If the question is ambiguous, the user compares described interpretations and chooses one (4) before anything runs. While the system works (5), it shows the current stage and attempt number; once the plan passes its checks, the supported operations and the risk class are previewed in the “Ask your data” card and the Working card before the read-only query runs (RC-18, RC-20). A released answer (6) shows how the question was read, the verification-support level, the result as chart and table, and up to three schema-checked follow-up questions (RC-21). Directly below it, two evidence cards (7) show the verification-support level with its reasons and a step-by-step correction replay (RC-17, RC-22), and the data used: sources, columns, rows considered and filters (RC-19). “How I got this” in the Data used card expands the risk-grouped checks, plan and SQL, so plain language comes first (RC-13). From the answer the user can mark it correct or flag an issue (10), open the history (9), or ask another question in the “Ask your data” card. Unsafe or unsupported requests stop at the capability check, and questions that still fail after three attempts reach the honest failure screen (8), which shows the reason and the attempts but no answer and no score.

Figure 2 separates the user roles. The business user (primary role) moves through screens 1–10. The data steward has no screens in the MVP: they edit semantic_layer.yaml offline, and those definitions appear read-only in the data overview (2) and in the evidence cards (7) (US-12, RC-05). Decision-makers do not use the app directly; they receive answers that can be traced back through the audit record. Once a dataset is loaded, a navy sidebar shows the current dataset and makes Ask, Data overview, History and Change dataset (screens 3, 2, 9 and 1) reachable from anywhere. The developer-only commands (US-13 evaluation and the optional US-14 baseline comparison) have no business-user screens. Screen numbers in Figure 2 carry through into the prototype and the implementation plan.

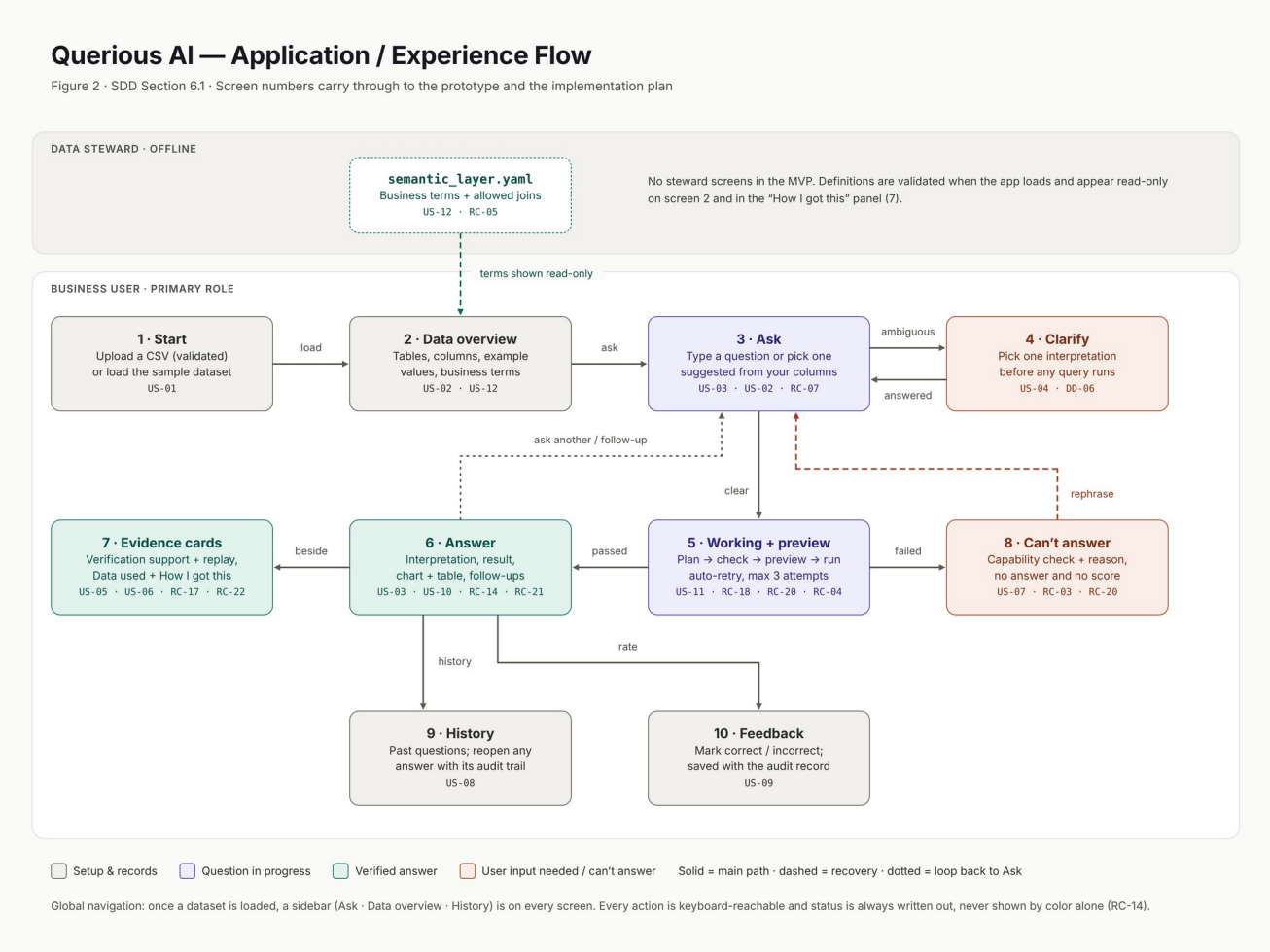


Figure 2: Application / experience flow for Querious AI, showing user roles, screens 1–10, the user actions that move between them, and the related US-## / RC-## items, including the v0.2 trust-layer states (Claude Design canvas, page "6.1 Experience flow").

6.2 Interactive UI/UX Prototype

The interactive prototype was generated with Claude Design using this SDD as context, refined after review (Section 6.3), and extended for SDD v0.2. It is a clickable desktop web-app prototype (RC-15) that implements screens 1–10 from Figure 2 with working state. Its layout is a Navy and Teal Workspace: a navy sidebar with the current dataset and navigation; an "Analysis workspace" with an "Ask your data" card that previews operations and risk, and an Answer card with the headline result, interpretation, chart and table; and, directly below the answer, two evidence cards side by side: "Verification support" (including correction replay) and "Data used" (provenance, with "How I got this" for checks, plan and SQL). Teal marks verified states and primary actions. Questions about the built-in sample dataset (fictional customers and orders, 2025–2026) follow scripted scenarios that cover every trust level and failure type. For an uploaded CSV, any typed question is turned into a query plan by a deterministic, schema-based planner that stands in for the LLM planner in the prototype. It matches column names, values from the file, dates, and measures such as total, average, count, highest and lowest, then computes the answer from the file in the browser, with SQL, checks, provenance and a dataset fingerprint. A ranking word such as "top" with no measure opens the clarification step, with choices built from the file's columns. A misspelled or approximate column name is fixed on a visible second attempt, and a question that matches nothing ends in the honest failure state. The prototype demonstrates the interface and its states; it is not evidence that the backend pipeline is implemented. Backend acceptance

checks such as the execution spy (US-07) and fixed signal fixtures (US-05, US-11) belong to the implementation tests. The written SDD remains authoritative where the prototype differs.

Prototype link (Claude Design, shareable): https://claude.ai/artifact/Afcj1f7hvk7mhgHfFhNtQJ

To use it, open the “6.2 Prototype (refined)” page and press Play on the “Querious prototype — Navy and Teal Workspace” artboard. It opens on a start screen where you can choose or drag in a CSV file or load the sample dataset. Two prototype-only controls are available under Tweaks: stage speed, and a switch that simulates the AI service being unavailable (RC-12).

Table 6.2: User flows demonstrated in the prototype

Flow	Steps in the prototype	Stories / requirements
A. Load data, explore and get suggestions	Start → “Load sample dataset” (or choose a non-CSV file to see the validation error) → Data overview with tables, column types, example values and steward-defined terms → Ask screen with three suggested questions built from this dataset. Uploading your own CSV changes the overview and suggestions to match its columns, and any question typed about the file is planned and answered from its rows (or honestly declined).	US-01, US-02, US-03, US-12, RC-07
B. Clarify → High verification support	Click “Who were our top customers last quarter?” → Clarify screen offers three described meanings, nothing preselected; Continue stays disabled until one is chosen and “None of these, rephrase” runs nothing → preview of operations and risk (High: declared join and clarification) → Answer with the chosen meaning, “High” support and its reasons, provenance, and follow-ups. The choice also appears in History.	US-04, US-05, US-11, RC-17, RC-18, RC-19, RC-20
C. Visible self-correction → Medium support and replay	Click “What was total revenue by region in 2025?” → attempt 1 fails validation (column region_name does not exist; nothing runs) → “Attempt 2 of 3” → preview → Answer with “Medium” support and “One correction required” → open “Replay correction” and step through both attempts: stage, outcome (“Not executed” vs “answer released”), error and plan change.	US-06, RC-04, RC-11, RC-17, RC-22
D. Non-blocking warning → Low support	Click the follow-up “Cancelled orders in 2025” → Medium-risk preview → answer released with “Low” support because 14 orders have a blank status; the warning is listed among the reasons and in the risk-grouped checks.	US-05, US-10, RC-17, RC-18
E. Honest failure	Type “What will sales be next quarter?” or “Delete all cancelled orders” → stopped at the capability check before any plan is compiled; or “Show me profit margin by supplier” → three failed validation attempts listed in the attempt replay. Each shows the reason, schema-valid suggestions, and no answer or score. The Tweaks switch shows the AI-service-unavailable state with Try again.	US-07, RC-01, RC-03, RC-04, RC-12, RC-20, RC-22
F. Follow-ups, feedback and history	On any answer, pick a follow-up (it runs as a new question through the full pipeline), mark it “Correct” or “Flag an issue”, then open History: each row shows status, support level, clarification choice and rating, and reopening restores the answer, provenance and replay without rerunning anything.	US-08, US-09, US-10, RC-09, RC-21, RC-22

6.3 Design Alignment & Human Refinement

Table 6.3a shows how the prototype supports the user stories and acceptance criteria in Section 3 and the trust-layer requirements in Section 4. US-13 and US-14 are developer commands with no business-user interface, so they are outside the prototype.

Table 6.3a: Requirements alignment

Story / req.	Where the prototype shows it	Acceptance criterion supported
US-01	Start screen: upload card checks file type, size and header row; sample dataset button.	An invalid file is rejected with a clear message; a valid dataset's tables and columns are displayed.
US-02, US-12	Data overview lists tables, column types, example values and the steward-defined terms (read-only); uploaded files show their own columns.	Every table, column and business term is listed with a plain-English description.
US-03	Ask screen with dataset-based suggestions; answers show a summary, chart and table.	Answers appear as a table and summary backed by the executed SQL shown in the audit panel.
US-04, RC-20	Clarify screen: three described meanings, none preselected, Continue disabled until chosen, rephrase exit runs nothing; the choice shows on the answer and in History.	Alternatives with definitions before any query; the selected meaning is used, displayed and retained.
US-05, RC-17, RC-19	Verification support and Data used cards beside every answer: High / Medium / Low with reasons (no percentages), sources, columns, rows considered and filters; "How I got this" expands the risk-grouped checks, plan and SQL.	Trust level and reasons, interpretation, terms, plan, risk class, SQL, checks, sources, columns, rows considered and filters are all shown.
US-06, RC-22	Region scenario: attempt 1 not executed, attempt 2 released; step-through replay from recorded events, reopened identically from History.	Each attempt's stage, error, plan change and outcome is shown; failed attempts never show a result.
US-07, RC-20	Forecast and delete requests stop at the capability check; missing-data questions fail after three attempts; no answer or score is shown.	Reason plus schema-valid rephrasing suggestions; early rejection before compilation and execution.
US-08, US-09	History rows show status, support level, clarification and rating; Correct / Incorrect buttons save to the audit record.	The rating is stored with the record and is visible when the answer is reopened.
US-10, RC-21	Up to three follow-ups on every answer, built from schema templates and the current plan; for uploads they change with the file's columns.	Only schema-valid suggestions are shown, and each runs through the full pipeline as a new question.
US-11, RC-18	After the plan is checked, a preview shows operations, risk class and extra checks before the user runs the query; "How I got this" groups checks by "every query", "Medium and High" and "High".	The risk class and its mandatory checks are visible before the answer is released.
RC-04, RC-11	Four named stages, "Attempt n of 3", and a note on whether anything has run on the data yet.	The current stage is always visible and the retry limit is explicit.

Story / req.	Where the prototype shows it	Acceptance criterion supported
RC-13, RC-14	Plain language in the main view; plan and SQL only inside “How I got this”; text and icons on every status; the underlying table one click below every chart (“View the underlying table”); real buttons, labels and focus styles.	Plain language first; status is never shown by color alone; every chart has its table.

Table 6.3b lists the refinements made after reviewing the first AI-generated draft (first six rows), one requested after reviewing the refined prototype, two made when the SDD was revised to v0.2, and one layout change chosen from three options (last three rows). The first draft is kept unchanged on the canvas page “6.3 AI draft v1” for side-by-side comparison.

Table 6.3b: Human refinements to the AI-generated design

Design refinement	Why it was changed	Related story / requirement
Completed the “How I got this” panel with the interpretation and clarification choice, business terms, plan, attempts and corrections, SQL executed, checks, and a header with the audit record ID, time, attempts and model.	The draft panel had no interpretation section, so it would fail the US-05 pass/fail check. RC-09 also requires each attempt, its error and the model to be traceable.	US-05, US-06, RC-09
Moved the interpretation and business terms into a visible “How I read your question” card above the result, with the clarification choice and a “Not what you meant? Rephrase” action.	In the draft the interpretation was a small grey line under the summary and the terms were hidden in the panel. RISK-04 (over-trust) requires both to stay visible in the main answer view.	US-04, US-12, RC-13, RISK-04
Redesigned the clarification step: bare option buttons became radio options that state each definition, with the note “No query has been run yet”, a Continue button and a “None of these, rephrase” exit.	The draft options gave no definitions and no way out, which pushed users toward guessing, against DD-06 (ask rather than guess). The exact choice is now shown on the answer and saved in the audit record.	US-04, DD-06, RC-09
Added an “Attempt n of 3” counter, numbered status icons with text, and a “Nothing has been run on your data yet” note to the Working screen.	The draft showed only colored dots, so status relied on color, and the retry loop was invisible until the answer appeared.	RC-04, RC-11, RC-14, US-06
Put the chart and its table in one Result card, printed values on the bars, and added text and icons to the status badges.	In the draft, opening the audit panel pushed the table away from its chart, and badges relied on color.	RC-14, US-03
Added a “Querious can / can’t” scope hint on the Ask screen, an AI-service-unavailable state with Try again, “What I tried” and “No numbers are shown” on the failure screen, and status and rating labels in History.	The draft had no RC-12 state, did not explain why no numbers were shown, and gave users no idea what was out of scope before they asked.	US-07, US-08, US-09, RC-01, RC-03, RC-12

Design refinement	Why it was changed	Related story / requirement
Made the suggested questions adapt to the loaded dataset: generated from its column names, types and business terms, each showing what it uses and checked against the schema. For an uploaded CSV the prototype reads the header row, infers column types, shows that file in the data overview, and answers suggested and typed questions from the file's rows.	Requested after reviewing the refined prototype: the example questions were fixed to the sample retail data, so a user who uploaded a different file would be told to ask about "top customers", which their data cannot answer. Suggestions use only schema metadata and business terms, never data rows.	US-02, US-03, US-10, RC-07, RC-21
Added a "Verification support" card (High / Medium / Low with the exact reasons and a note that it is not a probability) next to "How I read your question", a provenance strip under the result, and full provenance with a dataset fingerprint in the audit panel.	After the SDD was revised to v0.2, the prototype only showed a "checks passed" badge, which implied certainty without saying what was checked or which rows were used.	US-05, RC-17, RC-19, RISK-04
Added a capability and risk preview on the Working screen after the plan validates and before the query runs, risk-grouped checks in the audit panel, a step-through attempt replay, a capability line and attempt list on the failure screen, and schema-checked follow-ups on every answer.	SDD v0.2 requires users to see the work and its risk before execution, replay corrections from recorded events, and continue with valid follow-ups. The previous prototype showed corrections only as text and had no follow-ups.	US-06, US-07, US-10, US-11, RC-18, RC-20, RC-21, RC-22
Changed the layout to a Navy and Teal Workspace: a navy sidebar with the current dataset and navigation, an "Ask your data" card that shows the operations and risk before anything runs, an Answer card with a one-line headline, "How I read your question", the chart and the underlying table, and two evidence cards directly below the answer: "Verification support" (with correction replay) and "Data used" (with "How I got this"). Teal marks verified states and primary actions.	Chosen after comparing three layout options on the canvas (analyst workbench, conversation thread, answer report). The evidence first sat behind a button, then in a narrow right-hand column that squeezed the answer and its chart. I moved it into two cards directly below the answer, so the answer gets the full width and is read first, followed by why it can be trusted, with nothing to click. Keeping the evidence always visible supports RISK-04 and shows the main difference from a plain LLM; the operations-and-risk line in the Ask card makes RC-20 visible; SQL stays inside "How I got this" so plain language comes first.	US-05, US-06, RC-13, RC-17, RC-19, RC-20, RC-22, RISK-04

7 Key Design Decisions & Open Questions

7.1 Key Design Decisions

- **DD-01 - Constrained Pipeline Instead of an Autonomous Agent:** The LLM is limited to the ambiguity check and query planning; all execution, validation, and explanation are handled by deterministic code. This trades open-ended flexibility for correctness, consistency, and auditability, which matter most in data analysis.
- **DD-02 - Structured Query Plan (IR) Instead of Raw SQL:** The LLM produces a JSON plan matching a Pydantic schema rather than writing SQL. This allows validation before execution, deterministic SQL generation, and plain-English explanation of the plan.

- **DD-03 - Semantic Layer as a Version-Controlled File:** Business terms, metrics, and allowed joins are defined once in YAML by a data steward. This removes guesswork about business meaning and prevents incorrect joins.
- **DD-04 - Explanations Generated From the Plan:** User-facing explanations are built from the validated plan and actual results using templates, so the explanation cannot differ from what was executed.
- **DD-05 - Signal-Driven, Bounded Self-Correction:** Retries are triggered only by concrete signals (schema validation errors, database errors, failed sanity checks) and are capped at three. This avoids relying on the LLM to judge its own answer, which is known to be unreliable without external feedback.
- **DD-06 - Ask Rather Than Guess:** Ambiguous questions trigger a clarification step before any query runs.
- **DD-07 - Embedded, Local, Single-User MVP:** DuckDB and SQLite run in-process on the local machine, removing database servers, accounts, and hosting from the MVP and keeping user data local except for schema metadata sent to the LLM.
- **DD-08 - Observable Trust Rather Than Model Confidence:** Use the fixed RC-17 decision policy and plain-language reasons. This adds transparency without claiming that passing checks proves semantic correctness. Non-blocking warnings remain visible; mandatory failures prevent release.
- **DD-09 - Risk-Based Verification Within the Same IR:** Derive risk from supported plan features and clarification history, and use a small versioned registry of applicable checks. More verification strengthens declared joins and aggregations without adding unsupported SQL features or autonomous investigation.
- **DD-10 - Provenance and Replay From Audit Evidence:** Extend existing execution/audit records instead of asking the LLM to reconstruct what happened. Template-based follow-ups stay schema-valid and use the existing pipeline. No additional service or model role is needed.
- **DD-11 - Separate Research Comparison:** A guarded developer runner compares direct LLM-to-SQL with Querious. This isolates the raw-SQL experimental condition from normal user safety guarantees and keeps baseline work optional after core acceptance checks pass.

7.2 Open Questions, Assumptions & Risks

ID / Topic	Open Question, Assumption, or Risk
OQ-01 - LLM Provider and Model	Anthropic Claude is the leading option, but the final model has not been chosen. Compare candidate models on the gold question set for accuracy, latency, and cost before committing. The provider interface (RC-16) keeps this decision reversible.
OQ-02 - Query Plan Coverage	Which question types must the IR support for the MVP? The current assumption is filters, aggregations, grouping, sorting, top-N, time bucketing, and declared joins. Confirm coverage against the gold question set; unsupported questions must fail honestly.
OQ-03 - Gold Question Set and Dataset	Which dataset(s) will be used for evaluation, and how many questions? Assumption: one realistic sample dataset with 40-50 questions labeled as answerable, ambiguous, or unanswerable, possibly supplemented by a subset of a public text-to-SQL benchmark.

ID / Topic	Open Question, Assumption, or Risk
OQ-04 - Baseline Comparison	The baseline is a single direct LLM-to-SQL generation, not an unrestricted autonomous agent. Freeze the same model/version, decoding settings, schema, semantic definitions, allowed example values, dataset snapshot, questions, and resolved clarification inputs. Give the baseline no IR compiler or corrective feedback. Use the same safety envelope and result limits. Report first-attempt Querious and full bounded-correction Querious separately so extra calls are visible; an equal-call-budget baseline/ablation is future work. Invalid or rejected SQL counts as a failure, not an omitted case.
OQ-05 - Example Values and Privacy	Should example column values be sent to the LLM by default? They improve accuracy for filters but share some real data. The current assumption is off by default.
ASSUMPTION-01 - Data Shape	Datasets are clean, tabular CSV files with header rows, English column names, and sizes up to 50 MB.
RISK-01 - Ambiguity Detection Balance	The system may ask for clarification too often (annoying) or too rarely (guessing). Measure clarification precision on the gold set and tune prompts or thresholds.
RISK-02 - Sanity Check Errors	Result checks may flag correct results or miss wrong ones. Start with a small set of high-confidence rules and track false positives during evaluation.
RISK-03 - API Latency, Cost, and Rate Limits	Retries multiply LLM calls and may push response time past the 20-second target or exceed free-tier limits. Track calls per question in the evaluation report.
RISK-04 - Over-Trust	Passing checks does not prove that an interpretation or dataset is correct. Keep meaning and business definitions visible; call the ordinal trust score verification support rather than correctness probability. Measure wrong answers assigned High and warning/check false positives using gold-set truth.
FUT-01 - Pinned Answer Board (Future Work)	A board of pinned, verified answers is deferred beyond the MVP. Only released answers with a verification-support level could be pinned. Opening the board would rerun each tile's stored validated plan through the deterministic compiler, checks and read-only executor with no LLM call (RC-02, RC-06), recompute and display the support level and provenance (RC-17, RC-19), and flag tiles whose dataset fingerprint changed since pinning. Selecting a tile would open its full answer and audit trail. Scheduling, sharing, layout editing and BI export would remain out of scope. Adopting it would require narrowing the Section 2.3 non-goal, a new user story and requirement, a new screen in Figure 2, and prototype and evaluation updates.

Evaluation plan (US-13 core; US-14 optional research): Freeze one realistic sample dataset and 40–50 questions labeled answerable, ambiguous, or unsupported, with gold plans/results and risk classes. Include missing-column, date-boundary, null, join-multiplication, empty-result, truncation, and correction-limit cases. Validate the expected answers independently of Querious. Unit fixtures verify every branch of the trust policy and exact provenance counts; UI acceptance checks verify choice, preview, replay, and history fidelity.

For execution accuracy, compare normalized result sets to gold results (order only where required; document numeric tolerances) and include failures in the answerable denominator. Report clarification precision/recall on ambiguous and clear cases; score downstream accuracy separately after the same gold clarification is supplied. Correction success uses initially failed recoverable questions as its denominator. Run each answerable question three times and report result agreement, not merely SQL-

string equality. Stratify correctness and check failures by risk; report mandatory-check coverage, warning false positives, and incorrect High-support answers.

The optional paired baseline report uses identical frozen inputs and the RC-23 safety envelope. Report direct LLM-to-SQL, first-attempt Querious, and full bounded-correction Querious side by side: accuracy, invalid/rejected SQL rate, repeated-run agreement, median/p90 latency, calls, tokens, and estimated cost where available. Separate unsupported-request refusal from answerable accuracy. Publish raw per-question outcomes and counts; for the small gold set, results are descriptive and do not establish general superiority. Additional benchmark datasets, confidence calibration, and equal-budget ablations are future work.

8 References & Supporting Links

Anthropic. (n.d.). *Claude documentation*. Retrieved October 6, 2026, from https://docs.claude.com/

CSCI 5520 AI-Driven Software Development. (2026). *Homework/Lab 1: Software design and AI-led UI/UX* [Unpublished assignment instructions].

DuckDB Foundation. (n.d.). *DuckDB documentation*. Retrieved October 6, 2026, from https://duckdb.org/docs/

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). *Large language models cannot self-correct reasoning yet* [Conference paper]. The Twelfth International Conference on Learning Representations. https://arxiv.org/abs/2310.01798

Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., Zhou, X., Ma, C., Li, G., Chang, K. C.-C., Huang, F., Cheng, R., & Li, Y. (2023). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs. *Advances in Neural Information Processing Systems, 36*. https://bird-bench.github.io/

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., & Clark, P. (2023). Self-Refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2303.17651

Mao, T. (n.d.). *SQLGlot* [Computer software]. GitHub. https://github.com/tobymao/sqlglot

Pourreza, M., & Rafiei, D. (2023). DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2304.11015

Pydantic. (n.d.). *Pydantic documentation*. Retrieved October 6, 2026, from https://docs.pydantic.dev/

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2303.11366

World Wide Web Consortium. (2018). *Web content accessibility guidelines (WCAG) 2.1* [W3C Recommendation]. https://www.w3.org/TR/WCAG21/

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing reasoning and acting in language models* [Conference paper]. The Eleventh International Conference on Learning Representations. https://arxiv.org/abs/2210.03629

Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., & Radev, D. (2018). Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical*

Methods in Natural Language Processing (pp. 3911–3921). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1425